AI / ML & Data Science Interview Question & Answer Compendium

A consolidated, de-duplicated reference covering LLM/GenAI fundamentals, RAG, agents, LLMOps & production systems, classical ML, statistics, A/B testing, SQL, and coding — compiled from four separate interview-question sources.

Contents

1. LLM & GenAI Fundamentals

Q1. What is a Large Language Model (LLM)?

A neural network, almost always Transformer-based, trained on massive text corpora to predict the next token. Scale (parameters + data) gives it emergent abilities like reasoning, summarization, and few-shot task adaptation without task-specific training.

Q2. How are LLMs different from traditional language models?

Traditional models (n-gram, small RNNs) had limited context and were trained for one narrow task. LLMs use self-attention to model long-range dependencies, are trained on internet-scale data, and generalize across many downstream tasks via prompting instead of retraining.

Q3. What are Foundation Models?

Large models pretrained on broad data that serve as a general-purpose base, then adapted (via fine-tuning, prompting, or RAG) to many downstream tasks — e.g., GPT, Claude, Gemini, LLaMA.

Q4. How is GPT-4 different from GPT-3 in capabilities and applications?

GPT-4 added multimodal (image) input, longer context windows, stronger reasoning and instruction-following, and lower hallucination rates versus GPT-3, enabling more reliable use in coding, agents, and complex multi-step tasks.

Q5. What is tokenization, and how does it affect generation?

Tokenization splits text into subword units (e.g., BPE) that the model operates on. Vocabulary and merge rules affect sequence length, cost, and how well the model handles rare words, code, or non-English text — poor tokenization can fragment meaning and hurt generation quality.

Q6. What role do embeddings play in LLMs?

Embeddings map tokens (or documents) to dense vectors that capture semantic meaning, so similar concepts land close together in vector space. They're the input representation for the model and the basis for similarity search in RAG.

Q7. How do LLMs handle out-of-vocabulary (OOV) words?

Subword tokenization (BPE/WordPiece/SentencePiece) breaks unseen words into known subword pieces or even characters, so there's technically no true OOV — any string can be represented, just possibly at higher token cost.

Q8. What is a context window?

The maximum number of tokens (input + output) the model can attend to in a single call. Anything beyond it must be truncated, summarized, or retrieved on demand (RAG).

Q9. What is a hyperparameter?

A configuration value set before training/inference that isn't learned from data — e.g., learning rate, batch size, number of layers, temperature. Chosen via experimentation or search, not gradient descent.

Q10. An LLM generates offensive or incorrect outputs. How do you handle it?

Layer defenses: input/output moderation classifiers, RLHF/constitutional-AI alignment, retrieval grounding to reduce hallucination, guardrail prompts, human-in-the-loop review for high-risk outputs, and logging for post-hoc audit and retraining.

Q11. What are common challenges of using LLMs?

Hallucination, high inference cost/latency, limited context, non-determinism, prompt injection/security risks, bias, difficulty evaluating quality at scale, and keeping outputs current with fast-changing knowledge.

Q12. What is hallucination in LLMs, and how do you reduce it?

The model producing fluent but factually wrong or unsupported content, because it's optimized to predict plausible text, not verified truth. Mitigations: RAG grounding, citations, lower temperature, fact-checking/verifier passes, and fine-tuning on well-verified data.

Q13. Fine-tuning vs RAG vs Prompt Engineering — when do you choose each?

Prompt engineering first (cheapest, fastest iteration) for format/behavior control. RAG when the model needs current or proprietary knowledge without retraining. Fine-tuning when you need consistent style/format/domain behavior baked in, or to shrink/speed up a smaller specialized model — it's the most expensive and slowest to iterate on.

Q14. Encoder-only vs Decoder-only vs Encoder-Decoder — what's the difference?

Encoder-only (BERT) sees full bidirectional context — great for classification/embeddings. Decoder-only (GPT-family) is autoregressive, generating left-to-right — best for open-ended generation. Encoder-Decoder (T5, original Transformer) encodes input fully then generates output conditioned on it — good for translation/summarization where input and output are distinct sequences.

Q15. What is semantic caching?

Caching LLM responses keyed by the *meaning* of a query (via embedding similarity) rather than exact string match, so near-duplicate questions ('what's the capital of France' vs 'France's capital?') hit the cache and skip a costly model call.

Q16. Quantization — when should you quantize a model?

When you need to cut memory/compute for deployment (edge devices, cost-sensitive serving) and can tolerate a small accuracy loss. Techniques like INT8/INT4 or GPTQ/AWQ shrink weights; quantize once accuracy on your eval set stays within tolerance.

Q17. Latency vs Throughput in LLM serving — what's the difference?

Latency is time-to-response for one request (critical for chat UX); throughput is total requests/tokens served per unit time (critical for cost at scale). Batching improves throughput but can increase per-request latency — the core serving tradeoff.

Q18. What is speculative decoding?

A small 'draft' model quickly proposes several tokens ahead, and the large target model verifies them in a single parallel pass, accepting correct ones. This speeds up generation without changing output distribution, since verification falls back to the big model on mismatches.

Q19. Zero-shot vs one-shot vs few-shot learning — what's the difference?

Zero-shot: no examples, just an instruction. One-shot: exactly one example in the prompt. Few-shot: a handful of examples. More examples generally improve task-following/format accuracy but cost more tokens and context space.

Q20. Embedding model vs generation model — what's the difference?

An embedding model maps text to a fixed-size vector optimized for similarity/retrieval (not fluent text output). A generation model produces new text token-by-token. RAG systems typically use both: an embedding model for retrieval, a generation model for the final answer.

Q21. Streaming vs batch inference — what's the difference?

Streaming returns tokens incrementally as they're generated (better perceived latency for chat UIs). Batch inference processes many requests/prompts together for efficiency, better suited to offline jobs where turnaround time doesn't need to be sub-second.

2. Transformer Architecture, Pretraining & Decoding

Q1. What is attention in Transformer models?

A mechanism that lets each token compute a weighted combination of all other tokens' representations, where weights ('attention scores') reflect relevance, so the model can capture dependencies regardless of distance in the sequence.

Q2. What is Multi-head attention?

Running several attention operations ('heads') in parallel, each with its own learned projections, so different heads can specialize in different relationships (e.g., syntax vs. coreference). Outputs are concatenated and projected back to the model dimension.

Q3. How are attention scores computed in transformers?

Query and Key vectors are dot-producted, scaled by 1/sqrt(d_k) to stabilize gradients, then passed through softmax to get normalized weights, which are used to combine the Value vectors.

Q4. What is the softmax function and why is it used in attention?

It converts a vector of raw scores into a probability distribution (non-negative, sums to 1), so attention weights can be interpreted as 'how much to attend to each token' and combined smoothly and differentiably.

Q5. How is dot product used in self-attention?

The dot product between Query and Key vectors measures similarity/relevance between two tokens — a high dot product means the key token is important context for the query token, which becomes a large attention weight after softmax.

Q6. What are positional encodings in LLMs?

Since self-attention has no inherent notion of token order, positional encodings (sinusoidal, learned, or rotary/RoPE) inject position information into token representations so the model knows sequence order.

Q7. How does the Transformer architecture overcome challenges of traditional Seq2Seq models?

RNN-based Seq2Seq processes tokens sequentially, causing vanishing gradients and poor long-range memory, and can't parallelize over the sequence. Transformers use self-attention to directly connect any two positions and process the whole sequence in parallel, fixing both issues.

Q8. What are Sequence-to-Sequence (Seq2Seq) Models?

Models that map an input sequence to an output sequence of possibly different length, typically via an encoder that compresses input into a representation and a decoder that generates output from it — used for translation, summarization, etc.

Q9. What is the difference between an encoder and a decoder?

An encoder reads the full input and builds a contextual representation (often bidirectionally). A decoder generates output tokens one at a time, autoregressively, optionally attending to the encoder's output (cross-attention).

Q10. How do autoregressive models differ from masked models?

Autoregressive models (GPT) predict each token conditioned only on previous tokens (left-to-right), suited to generation. Masked models (BERT) see the whole sequence but predict randomly masked tokens using both left and right context, suited to understanding/representation tasks.

Q11. What is masked language modeling, and how does it help pretraining?

A pretraining objective where random tokens are replaced with a [MASK] and the model predicts them from bidirectional context. It forces the model to learn deep contextual representations useful for downstream classification/embedding tasks.

Q12. What is Next Sentence Prediction (NSP)?

A BERT pretraining task where the model predicts whether one sentence actually follows another in the original text, intended to teach sentence-level relationships (later shown to be less useful than MLM alone, and dropped in models like RoBERTa).

Q13. What is beam search, and how does it differ from greedy decoding?

Greedy decoding picks the single highest-probability token at each step, which can miss globally better sequences. Beam search keeps the top-k partial sequences ('beams') at each step and expands all of them, exploring more of the search space before committing to the final output.

Q14. Explain the concept of temperature in LLM text generation.

Temperature scales the logits before softmax. Low temperature (e.g., 0.1) sharpens the distribution toward the most likely tokens (more deterministic, repetitive); high temperature (e.g., 1.2) flattens it, increasing diversity and randomness.

Q15. What is the difference between top-k sampling and top-p sampling?

Top-k restricts sampling to the k highest-probability tokens regardless of their combined probability mass. Top-p (nucleus sampling) instead takes the smallest set of tokens whose cumulative probability exceeds p, so the candidate set size adapts to how peaked or flat the distribution is.

Q16. How does Adaptive Softmax speed up LLMs?

It groups vocabulary into frequency-based clusters, spending most compute on frequent tokens (short computation path) and less on rare ones (longer path), cutting the cost of the final softmax layer over huge vocabularies.

Q17. What is cross-entropy loss and why is it used in language models?

It measures the difference between the predicted token probability distribution and the true (one-hot) next token. It's used because it directly penalizes low probability assigned to the correct token and aligns training with maximum-likelihood next-token prediction.

Q18. How are gradients computed with respect to embeddings?

Via standard backpropagation: the loss gradient flows back through the network layers to the embedding lookup layer, updating only the rows of the embedding matrix corresponding to tokens that appeared in the batch.

Q19. What is the role of the Jacobian matrix in backpropagation?

The Jacobian captures how every output of a layer changes with respect to every input. Backprop's chain rule multiplies these Jacobians (or vector-Jacobian products, in practice) layer by layer to propagate gradients from the loss back to earlier parameters.

Q20. What is the chain rule and how is it used in deep learning?

It computes the derivative of a composed function by multiplying the derivatives of each component function. Backpropagation applies it repeatedly, layer by layer, to get the gradient of the loss with respect to every parameter in a deep network.

Q21. What is ReLU and why is it important?

ReLU(x) = max(0, x). It's cheap to compute, avoids the vanishing-gradient problem that sigmoid/tanh suffer for large inputs, and induces sparsity, which together made deep networks much easier to train.

Q22. What is the vanishing gradient problem, and how do Transformers solve it?

In deep/recurrent networks, gradients shrink exponentially as they propagate backward through many layers/time-steps, stalling learning in early layers. Transformers mitigate this with residual connections (skip paths that let gradients flow directly), layer normalization, and attention's direct token-to-token connections instead of long sequential chains.

Q23. What are eigenvalues and eigenvectors (in dimensionality reduction)?

For a matrix A, an eigenvector v satisfies $Av = \lambda v$, where λ is its eigenvalue. In PCA, the eigenvectors of the data's covariance matrix are the principal component directions, and the eigenvalues indicate how much variance each direction explains — so you keep the top eigenvectors to reduce dimensions with minimal information loss.

Q24. How is KL divergence used in evaluating LLMs?

KL divergence measures how much one probability distribution diverges from another. It's used to compare a model's output distribution to a reference (e.g., in RLHF to keep the policy close to the base model, or to detect distribution shift/drift between training and production outputs).

3. Fine-Tuning & Parameter-Efficient Methods (LoRA/QLoRA)

Q1. What is LoRA and QLoRA?

LoRA (Low-Rank Adaptation) freezes the pretrained weights and injects small trainable low-rank matrices into each layer, drastically cutting trainable parameters. QLoRA adds 4-bit quantization of the frozen base model on top, enabling fine-tuning of large models on much less GPU memory with minimal quality loss.

Q2. LoRA vs QLoRA vs full fine-tune — tradeoffs?

Full fine-tuning updates all weights — best quality ceiling but needs the most memory/compute and risks catastrophic forgetting. LoRA trains a small set of adapter weights — cheap, fast, easy to swap per task, slightly lower ceiling. QLoRA adds quantization to LoRA, cutting memory further at a small extra quality cost — best when GPU memory is the binding constraint.

Q3. What changes during fine-tuning? (optimizers, schedulers, layer freezing)

You typically use a lower learning rate than pretraining (to avoid destroying learned representations), a warmup + decay schedule, and may freeze earlier layers (which hold general features) while only training later layers or adapters — balancing adaptation speed against catastrophic forgetting.

Q4. How can catastrophic forgetting be mitigated in LLMs?

Use parameter-efficient methods (LoRA/adapters) instead of full fine-tuning, mix in a portion of the original pretraining/instruction data during fine-tuning, use lower learning rates, and apply regularization (e.g., keeping updates close to base weights, as in RLHF's KL penalty).

4. Generative vs Discriminative, Prompting Basics & RAG Concepts

Q1. What is overfitting, and how can it be prevented?

The model memorizes training data (including noise) instead of learning generalizable patterns, so it performs well on training data but poorly on unseen data. Prevent with more data, regularization (L1/L2, dropout), simpler models, early stopping, and cross-validation.

Q2. What are Generative and Discriminative models?

Generative models learn the joint distribution P(X, Y) (or just P(X)) and can generate new samples resembling the data (e.g., GPT, GANs, Naive Bayes). Discriminative models learn P(Y|X) directly to distinguish between classes (e.g., logistic regression, SVM) — usually better for pure classification but can't generate new data.

Q3. What is the difference between Discriminative AI and Generative AI?

Discriminative AI classifies or predicts labels for given input (fraud/not-fraud, spam/not-spam). Generative AI produces new content (text, images, code) by modeling how data is generated, e.g., LLMs and diffusion models.

Q4. How does prompt engineering influence the output of LLMs?

Since LLMs are highly sensitive to input phrasing, structure, and examples, prompt design directly shapes output format, tone, accuracy, and task success — without changing any model weights. Techniques include clear instructions, few-shot examples, role framing, and explicit output-format constraints.

Q5. What is Chain-of-Thought (CoT) prompting?

Prompting the model to explicitly reason step-by-step before giving a final answer (e.g., 'let's think step by step'), which measurably improves accuracy on multi-step reasoning, math, and logic tasks by giving the model intermediate computation to condition on.

Q6. What are the key steps in the Retrieval-Augmented Generation (RAG) pipeline?

1) Chunk and embed a knowledge base. 2) Store vectors in a vector DB/index. 3) At query time, embed the user query and retrieve top-k relevant chunks (often with reranking). 4) Insert retrieved chunks into the prompt as context. 5) Generate the answer grounded in that context, ideally with citations.

Q7. How does knowledge graph integration help LLMs?

Knowledge graphs provide structured, verifiable entity relationships that complement unstructured vector retrieval — enabling multi-hop reasoning ('who manages the manager of X'), reducing hallucination on relational facts, and giving explainable provenance for answers (GraphRAG).

Q8. How does Gemini's architecture improve training efficiency and stability compared to other multimodal LLMs?

Gemini was designed natively multimodal from the start (jointly trained on text, image, audio, video rather than bolting on modalities after the fact), and uses TPU-optimized distributed training infrastructure, which Google has described as improving training stability and efficiency at scale versus retrofitting unimodal architectures.

Q9. How does Mixture of Experts (MoE) improve scalability?

Instead of activating the entire network for every token, MoE routes each token to a small subset of specialized 'expert' sub-networks (via a learned gating function). This lets total parameter count scale up massively while keeping per-token compute (and thus inference cost) roughly constant.

Q10. What is zero-shot learning in LLMs?

The model performs a task purely from a natural-language instruction, with no task-specific examples provided in the prompt, relying entirely on knowledge learned during pretraining.

Q11. What is few-shot learning in LLMs and why is it useful?

Providing a handful of input-output examples directly in the prompt to demonstrate the desired task/format. It's useful because it steers behavior without any weight updates, works well for tasks with unusual formats, and is far cheaper/faster than fine-tuning.

5. Prompting & Context Engineering

Q1. Few-shot vs zero-shot — which works better where?

Zero-shot works well for simple, common tasks the model already understands well. Few-shot helps for unusual formats, domain-specific style, or ambiguous instructions, at the cost of extra tokens — a good default is to start zero-shot and add examples only if outputs miss the desired format.

Q2. How do you design system prompts that are robust across users?

Keep instructions explicit and structured (role, constraints, output format), separate immutable rules from user-editable content, add few-shot examples for edge cases, test against adversarial/ambiguous inputs, and version the prompt so behavior changes are tracked and reversible.

Q3. How do you make LLM output deterministic?

Set temperature to 0 (or near-0) and fix top-p/top-k, use a fixed random seed where the API supports it, and constrain output with structured formats (JSON schema / function calling) so even natural variation in phrasing doesn't change the extracted result. Full determinism isn't guaranteed across model versions or with server-side batching, so validate/re-check outputs downstream.

Q4. How do you track, version, and backfill changing context?

Store prompts/context templates in version control with semantic version tags, log which version produced each output, and run a backfill/replay job against historical inputs when context sources change, comparing old vs new outputs before rolling out broadly.

Q5. How do you build and maintain LLM memory?

Combine short-term memory (conversation buffer / rolling summary within the context window) with long-term memory (embeddings of past interactions stored in a vector DB, retrieved when relevant). Periodically summarize/compress old turns, and expire or downweight stale memories to avoid contradictions.

Q6. What is prompt compression?

Techniques (e.g., LLMLingua-style token pruning, summarization, or removing redundant context) that shrink prompt length while preserving task-relevant information, cutting token cost and latency, especially useful when injecting large retrieved contexts.

Q7. What is Context Engineering?

The broader discipline of designing what information — instructions, retrieved documents, tool outputs, memory, examples — goes into the model's context window, in what order and format, to maximize task success. It's prompt engineering generalized to dynamic, multi-source inputs (RAG, agents, tools).

Q8. What is Prompt Versioning?

Treating prompts as code: storing them with version numbers/git history, running regression evals when they change, and being able to roll back a prompt that degrades quality, just like a software deployment.

Q9. What are Structured Outputs?

Constraining the model to emit output in a strict schema (JSON, XML, function-call arguments) via grammar-constrained decoding or schema validation, so downstream code can parse the response

reliably instead of relying on fragile string parsing of free text.

Q10. What is the Needle-in-a-Haystack Test?

An evaluation that hides a specific fact ('the needle') at varying depths inside a long context ('the haystack') and checks whether the model can retrieve it, used to measure how well a model actually uses its full advertised context window rather than just accepting long inputs.

Q11. What is the 'Lost-in-the-Middle' problem?

Empirically, LLMs are more accurate at using information placed near the start or end of a long context and less accurate at using information buried in the middle — relevant to how you order retrieved chunks in a RAG prompt (put the most important ones first/last).

Q12. What is System Prompts & Security (prompt injection) about?

System prompts set the model's rules, but untrusted user or retrieved content can contain instructions trying to override them ('prompt injection'). Defenses include clearly delimiting untrusted content, instruction hierarchy/priority signaling, output filtering, and never treating retrieved/tool content as trusted instructions.

Q13. What is Constitutional AI?

An alignment technique (from Anthropic) where a model critiques and revises its own outputs against a written set of principles ('constitution'), then is fine-tuned on the improved responses, reducing reliance on large volumes of human-labeled harmful/safe examples.

6. RAG & Retrieval Engineering

Q1. What's your chunking strategy — by length, semantics, or structure?

Structure-aware chunking (splitting on headings, paragraphs, or code blocks) generally beats fixed-length splitting because it preserves semantic units; semantic chunking (splitting where embedding similarity drops) can do even better for unstructured prose. In practice, a hybrid — structural boundaries with a token-length cap and overlap — is a robust default.

Q2. How do you choose a vector DB (Chroma, Pinecone, OpenSearch, pgvector...)?

Weigh scale (millions vs billions of vectors), latency needs, existing infra (pgvector fits if you already run Postgres), managed vs self-hosted tradeoffs, filtering/metadata support, and cost. Lightweight/prototype use → Chroma/FAISS; production scale with managed ops → Pinecone; if you need hybrid SQL+vector in one system → pgvector.

Q3. Can you update or backfill embeddings with zero downtime?

Write new vectors to a shadow/parallel index while the old one keeps serving reads, validate quality on the new index, then atomically swap the alias/pointer to it — a blue-green deployment pattern applied to vector indexes.

Q4. How do you evaluate retrieval quality (precision@k, reranking, citation)?

Use precision@k / recall@k / MRR against a labeled query-document relevance set, check whether reranking improves the ordering of true positives toward the top, and verify that final answers actually cite (and are supported by) the retrieved chunks — not just topically related ones.

Q5. What are RAG failure modes?

Retrieval failures (wrong/no relevant chunks retrieved), generation failures (right context retrieved but model ignores or misreads it), stale or contradicting documents, chunking that splits key facts across boundaries, and over-long contexts triggering the lost-in-the-middle effect.

Q6. What is Hybrid Search?

Combining sparse lexical search (BM25/keyword) with dense vector similarity search, then merging/reranking results — captures both exact keyword matches (IDs, names) and semantic similarity, which either method alone often misses.

Q7. What are RAGAS metrics?

An evaluation framework for RAG systems measuring dimensions like faithfulness (is the answer supported by retrieved context), answer relevance, context precision, and context recall — letting you separately diagnose retrieval vs generation problems.

Q8. What is Multi-hop RAG architecture?

A retrieval pipeline that answers questions requiring chaining facts across multiple documents — retrieve, extract an intermediate fact, issue a follow-up retrieval based on it, and repeat — instead of a single retrieve-then-generate pass.

Q9. What is Multi-tenant RAG?

A RAG system serving multiple customers/tenants from shared infrastructure while strictly isolating each tenant's documents from others' retrieval results, typically via per-tenant namespaces/metadata filters in the vector index and enforced access control at query time.

Q10. How do you handle ambiguous queries in RAG?

Detect ambiguity (e.g., via retrieval score spread or explicit classifiers), then either ask a clarifying question, retrieve broadly and let the model note the ambiguity, or retrieve for multiple interpretations and let the answer address each.

Q11. How would you design a Legal Document Q&A System?

Chunk by clause/section (structure-aware), use hybrid search plus a reranker for precision, require citation of exact source clauses, add a verification/consistency pass since legal answers demand high faithfulness, and log every answer with its supporting evidence for audit.

Q12. HNSW vs IVF — what's the difference?

Both are approximate nearest-neighbor index structures. HNSW builds a multi-layer navigable graph — very fast, high recall queries, but higher memory and slower build time. IVF partitions vectors into clusters and searches only the nearest clusters — more memory-efficient and faster to build, typically at somewhat lower recall for the same speed.

Q13. What is Prompt Compression? (RAG context)

See prompting section — in RAG specifically, it means compressing retrieved passages before insertion so more distinct sources fit within the context budget without exceeding token limits.

Q14. What are Knowledge Graphs & GraphRAG?

GraphRAG retrieves not just similar text chunks but also structured entity-relationship subgraphs, letting the model answer relational and multi-hop questions more reliably and with traceable provenance, versus purely vector-similarity retrieval.

Q15. What is Metadata Filtering in retrieval?

Restricting vector search to a subset of documents matching structured metadata (date range, tenant, document type, permission level) before or alongside similarity search, improving both relevance and access control.

Q16. Reranker vs Retriever — what's the difference?

The retriever does a fast, approximate first pass over the whole corpus (e.g., ANN vector search) to get a candidate set. The reranker is a more expensive, more accurate model (often a cross-encoder) that re-scores just that smaller candidate set for the final top-k ordering — a two-stage precision/speed tradeoff.

Q17. What is Retrieval-based Few-shot Prompting?

Instead of using a fixed set of few-shot examples, dynamically retrieving the most relevant examples for the current input (via embedding similarity) and inserting those into the prompt, improving relevance over static examples.

Q18. Document Q&A vs Conversational RAG — what's the difference?

Document Q&A typically answers one-off questions against a static corpus. Conversational RAG must additionally handle multi-turn context (resolving pronouns/follow-ups against prior turns) and often needs to rewrite the current turn into a standalone query before retrieval.

Q19. How do you handle contradicting documents in RAG?

Surface both perspectives with their sources rather than silently picking one, weight by recency/authority/metadata trust score, and explicitly instruct the model to flag conflicts instead of resolving them silently.

Q20. How do you prevent stale information in RAG?

Track document timestamps and prefer/boost recent sources, set TTLs or scheduled re-crawls for source data, invalidate/re-embed on source updates, and, where staleness matters most, explicitly surface 'as of' dates in the answer.

Q21. pgvector vs Dedicated Vector DB — what's the tradeoff?

pgvector adds vector search to existing Postgres — simpler ops, transactional consistency with your relational data, good for small/medium scale. Dedicated vector DBs (Pinecone, Milvus, Weaviate) are purpose-built for billions of vectors, offer more advanced indexing and horizontal scaling, at the cost of an extra system to run and keep in sync.

7. Agents & GenAI Frameworks

Q1. What is the ReAct pattern?

A prompting/agent framework interleaving Reasoning traces ('thought') with Actions (tool calls), then feeding the observation back in — letting the model plan, act, observe, and re-plan iteratively rather than answering in one shot.

Q2. What is LLM-as-Judge?

Using a (typically stronger) LLM to score or compare the outputs of another model against a rubric, as a scalable substitute for human evaluation — useful for automated regression testing of prompts/models, though it can inherit biases and needs periodic calibration against human judgments.

Q3. What is MCP (Model Context Protocol)?

A standardized protocol for connecting LLM applications to external tools, data sources, and services in a uniform way, so an assistant can discover and call e.g. a calendar, database, or ticketing system without a bespoke integration per tool.

Q4. What is vLLM?

An open-source high-throughput LLM serving engine, notable for PagedAttention (memory-efficient KV-cache management inspired by OS virtual memory paging) and continuous batching, which together significantly improve serving throughput versus naive batching.

Q5. How do you prevent infinite agent loops?

Set hard limits on iteration count/time/tokens, detect repeated identical actions or states (loop detection), require the agent to make measurable progress each step, and add a fallback/escalation-to-human path when limits are hit.

Q6. How do you evaluate an agent?

Beyond final-answer correctness: task completion rate, number of steps/tool calls to succeed, tool-call accuracy (right tool, right arguments), robustness to unexpected tool outputs/errors, and cost/latency per completed task.

Q7. What is DSPy?

A framework for programmatically optimizing prompts and few-shot examples for a pipeline of LLM calls, treating prompt engineering as a compilable/optimizable program rather than hand-tuned text, using metrics to automatically search for better prompts.

Q8. What is Attention Sink?

The empirical phenomenon where models allocate disproportionate attention weight to the first few tokens of a sequence (regardless of their content), which turns out to be important for maintaining stability in streaming/long-context generation — removing them can degrade quality.

Q9. LangChain vs LangGraph — what's the difference?

LangChain provides composable building blocks (chains, retrievers, tool wrappers) for linear or simple branching pipelines. LangGraph models an agent's execution as an explicit state graph with cycles/conditional edges, better suited to complex multi-step agents that need loops, retries, and

branching control flow.

Q10. RLHF vs DPO — what's the difference?

RLHF trains a separate reward model from human preference data, then optimizes the policy against it using RL (e.g., PPO) — powerful but complex and unstable to tune. DPO reformulates the same preference-learning objective as a direct classification-style loss on the policy itself, skipping the separate reward model and RL loop — simpler and more stable, with comparable results in many settings.

Q11. What is OpenAI Agents SDK vs LangGraph vs CrewAI?

All are agent-orchestration frameworks with different philosophies: OpenAI's Agents SDK is a lightweight, provider-native way to define agents/handoffs/guardrails; LangGraph offers explicit graph-based control flow for complex stateful agents; CrewAI focuses on orchestrating multiple role-based agents collaborating as a 'crew.' Choice depends on how much control-flow complexity and multi-agent coordination you need versus wanting simplicity.

Q12. What is Chain-of-Verification?

A technique where the model first drafts an answer, then generates and answers independent verification questions about its own claims, and finally revises the draft based on any inconsistencies found — reducing hallucination versus a single-pass answer.

Q13. What is Agent Memory Staleness?

The risk that an agent's stored memory (facts, user preferences, past decisions) becomes outdated as the real world changes, causing it to act on wrong assumptions. Mitigated with memory TTLs, periodic revalidation, and timestamping stored facts.

Q14. LLM Gateway vs Orchestration Framework — what's the difference?

An LLM gateway is an infrastructure layer that provides a unified API across multiple model providers, handling auth, rate limiting, routing, caching, and cost tracking. An orchestration framework (LangGraph, CrewAI) sits above that, defining the actual multi-step agent logic and control flow — the gateway is about *which model answers*, the framework is about *what steps happen*.

Q15. What is Graceful Degradation (in LLM systems)?

Designing the system so that when a component fails (model timeout, tool error, retrieval empty), it falls back to a simpler but still useful behavior — a cached answer, a smaller model, a canned response, or a clear 'I can't complete this right now' — rather than failing the whole request.

Q16. How would you architect GenAI systems for concurrent users?

Use async/non-blocking request handling, connection pooling to model providers, request queuing with backpressure, horizontal scaling of stateless serving instances, caching (semantic + exact) to cut redundant calls, and separate rate-limit budgets per tenant to prevent noisy-neighbor issues.

Q17. How do memory, caching, retrieval, and latency fit together?

Memory determines what context needs to be assembled; caching avoids recomputing/re-fetching that context or re-generating identical answers; retrieval fetches the specific facts needed for this query; and all three exist specifically to keep end-to-end latency acceptable despite the cost of a full LLM call — each is a lever to reduce how much 'live' work the model has to do.

8. LLMOps, Production Systems & Security

Q1. Sketch a pipeline: from raw data → model → serving → feedback.

Ingest & clean raw data → label/curate (or chunk+embed for RAG) → train/fine-tune or configure prompts → offline eval against a golden set → canary/shadow deploy → serve behind a gateway with logging → collect user feedback/implicit signals → feed corrections back into the eval set and next training/prompt iteration.

Q2. How would you monitor performance drift or hallucinations in production?

Track output-quality proxies over time (LLM-as-judge scores on a sample, citation/groundedness rate, user thumbs-down rate, retrieval-hit rate), alert on statistically significant shifts versus a baseline window, and periodically re-run the full eval suite against production traffic samples.

Q3. How do you log prompts and outputs for debugging and auditing?

Log the full resolved prompt (including retrieved context and template version), model/version used, parameters (temperature, etc.), raw output, latency, and cost per request, with request IDs for tracing — while redacting/encrypting PII in line with data-retention policy.

Q4. CI/CD for LLM workflows — what's different from traditional ML?

Traditional ML CI/CD gates on model metrics from a fixed test set. LLM CI/CD must also version prompts/templates (not just weights), run non-deterministic outputs through statistical/LLM-judge evals rather than exact-match tests, and test against adversarial/edge-case prompts — plus rollback needs to cover prompt versions, not only model versions.

Q5. Why don't traditional ML metrics work for LLMs?

Traditional metrics (accuracy, F1) assume a single correct discrete label. LLM outputs are open-ended free text where many different phrasings can be equally correct, so you need semantic-similarity, faithfulness/groundedness, or LLM-judge-based metrics rather than exact match.

Q6. How do you detect prompt drift before users notice?

Continuously run a fixed regression suite of representative prompts against the live system and diff outputs/scores against a known-good baseline on every prompt or model change, alerting on any significant score drop before it reaches broad production traffic.

Q7. How do you test a non-deterministic system?

Run each test case multiple times and assert on distributions/statistics (e.g., 95% of runs meet the bar) rather than a single exact output, use semantic-similarity or LLM-judge scoring instead of string equality, and pin temperature/seed for tests where determinism is achievable.

Q8. What should you version besides the model?

Prompt templates, retrieval index/embedding model version, tool/function definitions, system configuration (temperature, top-p), and the eval dataset itself — since any of these changing can change behavior as much as swapping the model.

Q9. Why is retraining usually the last option in LLMOps?

It's the slowest and most expensive lever — cheaper options (prompt tweaks, RAG updates, few-shot examples, guardrails) can fix most issues faster and with less risk of regressions elsewhere; retraining is reserved for cases where the base model's fundamental knowledge or behavior genuinely needs to change.

Q10. How do you separate retrieval failures from generation failures?

Inspect the retrieved context independently of the final answer: if the right facts were retrieved but the answer is still wrong, it's a generation failure; if the retrieved context lacks the needed facts, it's a retrieval failure. RAGAS-style metrics (context recall vs faithfulness) formalize this split.

Q11. When do you choose a chatbot, RAG, or an AI agent?

Chatbot: conversational Q&A with knowledge the model already has or that fits in-context. RAG: answers must be grounded in specific, possibly large or changing external knowledge. Agent: the task requires multi-step planning and taking actions via tools (not just answering), e.g., booking, executing workflows, or coordinating multiple systems.

Q12. What is the OWASP LLM Top 10?

A community-maintained list of the most critical LLM application security risks — including prompt injection, insecure output handling, training data poisoning, model denial-of-service, supply-chain vulnerabilities, sensitive information disclosure, insecure plugin/tool design, excessive agency, overreliance, and model theft.

Q13. How do you design Low-Latency LLM Systems?

Use smaller/distilled models or speculative decoding where possible, stream tokens to the client immediately, cache aggressively (semantic + exact), keep retrieval fast (approximate NN indexes), colocate services to cut network hops, and parallelize independent sub-tasks (e.g., retrieval while a smaller model handles an initial response).

Q14. How do you handle API Failures & Rate Limits from model providers?

Implement retries with exponential backoff and jitter, circuit breakers to stop hammering a failing provider, request queuing with graceful shedding, and multi-provider fallback (route to a secondary model/provider) so a single provider outage doesn't take down the whole system.

Q15. How would you investigate a sudden user satisfaction drop?

Check for recent deploys (prompt/model/config changes) around the drop's onset, segment complaints by feature/user cohort to localize the issue, compare current output-quality metrics (groundedness, latency, error rate) against the prior baseline, and reproduce reported failures directly before rolling back or patching.

Q16. What are the tradeoffs of On-Prem LLM Deployment?

Pros: data never leaves your infrastructure (compliance/security), predictable cost at scale, no external rate limits. Cons: you own GPU procurement/scaling/ops, miss out on frontier hosted-model quality improvements, and need in-house MLOps expertise — generally chosen for regulated or highly sensitive data (e.g., defense, healthcare, finance).

Q17. Debugging scenario: a duplicate-email agent bug — how would you approach it?

Reproduce with the exact input/state that triggered duplicates, check for missing idempotency keys or retry logic re-triggering the send action, inspect whether the agent's loop/step tracking allowed the same action to be selected twice, add idempotency guards on the send action itself, and add a regression test for the exact scenario.

Q18. How do you reduce LLM costs in production?

Cache aggressively, route easy queries to a cheaper/smaller model and only escalate hard ones to the frontier model, shorten prompts (compression, better chunking), batch requests, use quantized/distilled models where quality allows, and set token budgets/limits per request type.

Q19. Coding: implement exponential backoff for API retries.

```
import time, random

def call_with_backoff(fn, max_retries=5, base_delay=1.0):
for attempt in range(max_retries):
try:
return fn()
except Exception:
if attempt == max_retries - 1:
raise
delay = base_delay * (2 ** attempt) + random.uniform(0, 0.5)
time.sleep(delay)
```

Q20. Walk through building a RAG pipeline end to end.

1) Ingest & clean docs. 2) Chunk (structure-aware, ~300-800 tokens, with overlap). 3) Embed chunks and upsert into a vector index with metadata. 4) At query time, embed the query, retrieve top-k (+ optional hybrid/reranking). 5) Assemble a prompt with retrieved context + citation instructions. 6) Generate the answer. 7) Log context, answer, and feedback for continuous eval.

Q21. How do you validate tool calls made by an agent?

Validate arguments against a strict schema before execution, apply allow-lists/permission scopes per tool, sandbox side-effecting actions where possible, require confirmation for irreversible actions, and log every call with inputs/outputs for auditing.

Q22. Tell me about a time you improved system performance. (behavioral)

Structure your answer with the STAR method: the baseline metric and bottleneck you identified, the specific change you made (e.g., caching, batching, a smaller model, better indexing), the measured before/after improvement, and what you'd do differently next time.

Q23. Quality vs Cost Tradeoff — how do you reason about it?

Define the minimum acceptable quality bar for the use case (not 'as good as possible'), then find the cheapest model/pipeline that clears that bar — often via a tiered routing approach (cheap model first, escalate only on low-confidence or high-stakes queries) rather than a single one-size-fits-all model choice.

Q24. Tell me about a production incident you handled. (behavioral)

Frame it as: what alerted you / how you detected it, how you triaged and mitigated impact quickly (rollback, feature flag, fallback), the root cause once found, and the concrete follow-up (monitoring, test, or process change) that prevents recurrence.

Q25. How do you stay current with fast-moving GenAI developments? (behavioral)

A genuine answer names specific habits — following key papers/changelogs (arXiv, provider release notes), hands-on experimentation with new tools/models, and a small personal project or internal PoC pipeline used to sanity-check new techniques before recommending them for production.

Q26. Open-Weight Models vs Frontier (closed) Models — tradeoffs?

Open-weight models (Llama, Mistral, Qwen) allow self-hosting, fine-tuning, and full data control at lower marginal cost, but generally lag frontier closed models (GPT, Claude, Gemini) on the hardest reasoning tasks and require you to own the serving infrastructure. Closed frontier models offer top capability and zero ops burden, at higher per-token cost and less control over data/customization.

Q27. How do you handle PII in LLM systems?

Detect and redact/mask PII before it's sent to third-party model APIs where required, minimize retention/logging of raw PII, encrypt at rest and in transit, apply access controls and audit logs to any stored personal data, and align retention windows with applicable regulations (e.g., GDPR).

Q28. How do you A/B test prompt changes safely?

Route a small percentage of traffic to the new prompt, hold model/other variables fixed, compare output-quality metrics (groundedness, user feedback, task completion) and cost/latency against the control, and only ramp up once the new prompt is statistically better (or at least non-inferior) with no safety regressions.

Q29. What is Model Distillation?

Training a smaller 'student' model to mimic the outputs (or output distribution) of a larger 'teacher' model, capturing most of its capability on the target task at a fraction of the inference cost and latency.

Q30. How do you handle traffic spikes in an LLM system?

Auto-scale stateless serving replicas, queue and gracefully shed excess load rather than failing hard, pre-warm caches for predictable spikes, apply per-tenant rate limits so one customer's spike doesn't starve others, and have a smaller/cheaper fallback model to absorb overflow if the primary model is saturated.

Q31. What would you put on a GenAI product metrics dashboard?

Task success/completion rate, groundedness/citation rate, latency (p50/p95), cost per request and per user, error/fallback rate, user feedback (thumbs up/down), and retrieval quality (hit rate) if RAG is involved — split by feature/segment where possible to catch localized regressions.

9. Cost, Latency & System-Design Reasoning

Q1. How do you reduce token usage?

Compress/trim retrieved context, summarize long conversation history instead of replaying it verbatim, use concise system prompts, avoid redundant few-shot examples, and cap max output tokens to what's actually needed.

Q2. When should you quantize a model? (duplicate — see Fundamentals section)

See 'Quantization' above under Fundamentals.

Q3. What's your batching + caching strategy to reduce latency?

Use continuous/dynamic batching at the serving layer to keep GPUs busy without hurting per-request latency much, combine with exact-match caching for repeated queries and semantic caching for near-duplicates, and prefetch/precompute for predictable request patterns.

Q4. When to use hosted APIs vs open-source models?

Hosted APIs (OpenAI, Anthropic, etc.) win when you want top capability, zero infra ops, and can tolerate per-token cost and external dependency. Self-hosted open-source models win when you need data residency/control, very high volume where marginal cost matters, or heavy customization via fine-tuning.

Q5. Can you solve this without an LLM or vector DB?

Always worth asking first — many problems (structured lookups, rule-based classification, exact keyword search) are solved faster, cheaper, and more reliably with traditional software (SQL queries, regex, simple heuristics) than by reaching for an LLM.

Q6. What's the right database for this task — SQL, NoSQL, or vector?

SQL for structured, relational data with strong consistency needs. NoSQL (document/key-value) for flexible schemas and high write throughput. Vector DB specifically when you need semantic similarity search over embeddings — often used alongside, not instead of, a relational store.

Q7. How do you make an AI system more deterministic and less brittle?

Constrain outputs with schemas/function calling, lower temperature, add validation/retry loops on malformed outputs, fall back to rule-based logic for critical decision points, and keep the LLM's role scoped to what genuinely needs language understanding rather than using it for tasks better solved deterministically.

Q8. What fallback do you use if the LLM fails mid-task?

Retry with backoff, fall back to a secondary model/provider, degrade to a simpler deterministic response or cached answer, and surface a clear, actionable error to the user rather than an opaque failure — logging the failure for follow-up.

Q9. What happens if your embedding model changes — how do you migrate safely?

Re-embed the entire corpus with the new model into a separate index (old and new embeddings aren't comparable), validate retrieval quality on the new index against a labeled eval set, then cut traffic over (blue-green), keeping the old index available briefly for rollback.

Q10. How would you fine-tune a model on user behavior and deploy it?

Collect and curate representative interaction data (with consent/privacy review), hold out an eval set, fine-tune (often via LoRA for efficiency), evaluate against both the eval set and known regression cases, canary-deploy to a small traffic slice, monitor quality/safety metrics, then ramp up.

Q11. How would you make this system cheaper without killing quality?

Route by difficulty (cheap model for easy queries, escalate only when confidence is low), cache aggressively, compress context, and quantize/distill where a smaller model clears the quality bar — validating each change against your eval suite before rollout.

Q12. Walk through a debugging session for incorrect LLM outputs.

Reproduce the failure with the exact prompt/context that was actually sent (not an approximation), check whether the correct information was even present in context (retrieval vs generation failure), test whether the issue persists at temperature 0, check for prompt-template bugs or truncated context, and compare against a known-good prompt/model version to isolate what changed.

Q13. How do you prevent FastAPI bottlenecks with LLM workloads?

Use async endpoints and async HTTP clients to the model provider so slow LLM calls don't block the event loop, run CPU-bound work (e.g., embedding, reranking) in worker processes/thread pools, add request queuing/concurrency limits, and scale horizontally behind a load balancer rather than relying on a single process.

10. Classical Machine Learning

Q1. What are the main types of Machine Learning?

Supervised (learns from labeled input-output pairs, e.g. spam detection), Unsupervised (finds structure in unlabeled data, e.g. clustering), Semi-supervised (small labeled + large unlabeled set, useful when labeling is costly), and Reinforcement Learning (an agent learns from rewards/penalties via environment interaction, e.g. game AI, robotics).

Q2. What is the difference between Classification and Regression?

Classification predicts a discrete category/label (spam vs not-spam). Regression predicts a continuous numeric value (house price, temperature).

Q3. What is Overfitting vs Underfitting?

Underfitting: the model is too simple to capture patterns, performing poorly on both train and test data. Overfitting: the model memorizes training data including noise, performing well on train but poorly on unseen data. Fix underfitting with a more complex model/more features; fix overfitting with more data, regularization, simpler models, or cross-validation.

Q4. What is the Bias–Variance tradeoff?

Bias is error from overly simplistic assumptions (leads to underfitting); variance is error from being overly sensitive to the specific training set (leads to overfitting). Increasing model complexity typically lowers bias but raises variance, and vice versa — a good model balances both to minimize total error on unseen data.

Q5. What is Cross-validation, and why is it important?

A technique to reliably estimate how well a model generalizes by splitting data into multiple folds, training on some and testing on the rest, repeating, and averaging results — rather than trusting a single train-test split. K-Fold is the most common variant; it uses data more efficiently and helps detect overfitting.

Q6. How do you evaluate a classification model?

Beyond raw accuracy (misleading on imbalanced data): precision (of predicted positives, how many were correct), recall (of actual positives, how many were caught), F1-score (harmonic mean balancing both), and the full confusion matrix to see the error breakdown. Which matters most is task-dependent — e.g., recall matters more than accuracy in fraud detection, where missing a fraud case is costly.

Q7. What is the ROC curve and AUC?

The ROC curve plots true positive rate vs false positive rate across classification thresholds. AUC (Area Under the Curve) summarizes it into one number representing how well the model separates classes — close to 1 is strong, close to 0.5 is random guessing. Especially useful for imbalanced classification problems.

Q8. What is Feature Scaling, and why does it matter?

Ensures numerical features are on comparable scales so no feature dominates purely due to magnitude, which matters for distance- or gradient-based algorithms (KNN, SVM, gradient descent). Normalization rescales to [0,1]; standardization centers data to mean 0, unit variance. Tree-based models generally don't need scaling since splits are scale-invariant.

Q9. How does a Decision Tree make decisions? When would you use it over other algorithms?

At each node it picks the feature and threshold that best splits data into purer subsets, using Gini impurity or entropy as the split criterion. Easy to interpret and handles mixed numerical/categorical data, but prone to overfitting and unstable to small data changes — use it when interpretability matters more than raw accuracy, or as a building block for ensembles.

Q10. What is a Random Forest, and why is it better than a single Decision Tree?

An ensemble of many decision trees, each trained on a random data subset and random feature subset, combined via majority vote (classification) or averaging (regression). Different trees' mistakes cancel out, reducing overfitting and improving stability versus one tree, at the cost of interpretability and more memory/compute.

Q11. What is the difference between Bagging and Boosting?

Bagging (e.g., Random Forest) trains many models in parallel on bootstrapped samples and averages them to reduce variance. Boosting (e.g., XGBoost, AdaBoost) trains models sequentially, each one focused on correcting the previous model's errors, reducing bias — generally higher accuracy potential but more prone to overfitting and harder to parallelize.

Q12. What are L1 and L2 Regularization, and why use them?

Both add a penalty on model weights to discourage overly large coefficients and reduce overfitting. L1 (Lasso) penalizes the absolute value of weights and can shrink some to exactly zero, performing implicit feature selection. L2 (Ridge) penalizes squared weights, shrinking them smoothly but rarely to zero, keeping all features with reduced impact.

Q13. What is Gradient Descent, and how does it work?

An optimization algorithm that minimizes a loss function by iteratively: computing the loss, computing its gradient with respect to parameters, and stepping parameters in the opposite direction of the gradient, repeating until convergence. Learning rate controls step size — too high risks never converging, too low is very slow.

Q14. What is Stochastic Gradient Descent (SGD) vs Batch Gradient Descent?

Batch gradient descent computes the gradient over the entire dataset before each update — accurate but slow and memory-heavy for large datasets. SGD updates parameters using one (or a small mini-batch of) example(s) at a time — much faster per step and can escape shallow local minima, at the cost of noisier convergence.

Q15. Why is the Train–Validation–Test split important?

It lets you honestly evaluate generalization: the training set teaches the model, the validation set tunes hyperparameters and model choices, and the test set — used only once — gives an unbiased final performance estimate. Proper splitting prevents data leakage and overstated performance claims.

Q16. What is a Confusion Matrix, and how do you read it?

A table of actual vs predicted classes with four cells: True Positive, True Negative, False Positive (predicted positive, actually negative), and False Negative (predicted negative, actually positive). Accuracy, precision, recall, and F1 are all derived from it — it reveals the specific types of errors a model

makes, not just an aggregate score.

Q17. What is Feature Engineering, and why is it important?

Creating better input representations for a model — e.g., extracting day/weekday from a date, encoding categorical variables — since models learn from features, not raw data. Good features often improve performance more than changing the algorithm itself.

Q18. What is Supervised vs Unsupervised learning?

Supervised learning trains on labeled input-output pairs to predict labels for new data. Unsupervised learning works with unlabeled data to discover structure — e.g., clustering, dimensionality reduction — with no 'correct answer' to check against.

Q19. How do you handle missing or corrupted data in a dataset?

Diagnose the missingness pattern (random vs systematic), then choose: drop rows/columns if missingness is small and non-informative, impute (mean/median/mode, or model-based imputation) if data is valuable, or add a 'missing' indicator flag when the fact of missingness itself is informative. For corrupted data, validate against expected ranges/types and correct or discard outliers/malformed records.

Q20. Can you explain the principle of a Support Vector Machine (SVM)?

SVM finds the hyperplane that maximizes the margin (distance) between classes, using only the closest points ('support vectors') to define that boundary. Kernel functions (RBF, polynomial) let it separate non-linearly-separable data by implicitly mapping to a higher-dimensional space.

Q21. What are the advantages and disadvantages of a Neural Network?

Advantages: can model highly complex, non-linear relationships and scale with more data; state-of-the-art on unstructured data (images, text, audio). Disadvantages: needs large data and compute, is less interpretable ('black box'), prone to overfitting without regularization, and requires more careful tuning than simpler models.

Q22. How does the k-means algorithm work?

Choose k initial cluster centroids, assign each point to its nearest centroid, recompute centroids as the mean of assigned points, and repeat until assignments stabilize. Sensitive to initial centroid placement and requires choosing k in advance (e.g., via the elbow method).

Q23. What is Principal Component Analysis (PCA), and when is it used?

A dimensionality-reduction technique that projects data onto the directions (principal components) of maximum variance, computed from the eigenvectors of the data's covariance matrix. Used to reduce feature count while retaining most information, speed up downstream models, and visualize high-dimensional data.

Q24. What is an activation function, and why is it used in a neural network?

A non-linear function (ReLU, sigmoid, tanh) applied to a neuron's output. Without it, stacking linear layers would collapse to a single linear transformation regardless of depth — activation functions are what let networks model non-linear relationships.

Q25. How would you handle an imbalanced dataset?

Resampling (oversample the minority class — e.g., SMOTE — or undersample the majority), class-weighted loss functions, choosing threshold-independent metrics (AUC, PR-AUC) instead of raw accuracy, and ensemble methods designed for imbalance (e.g., balanced random forest).

Q26. Can you explain the concept of feature selection in machine learning?

Choosing the subset of input features most relevant to the target, to reduce overfitting, training time, and noise. Methods include filter methods (correlation/statistical tests), wrapper methods (recursive feature elimination), and embedded methods (L1 regularization, tree-based feature importance).

Q27. Can you describe how a convolutional neural network (CNN) works?

CNNs apply learned filters (kernels) that slide over the input to detect local spatial patterns (edges, textures, shapes), with pooling layers downsampling to build translation-invariant, hierarchical features — early layers detect simple patterns, deeper layers detect more complex/abstract ones, well suited to image and spatial data.

Q28. How do you handle categorical variables in your dataset?

One-hot encoding for low-cardinality nominal categories, ordinal encoding when categories have a natural order, and target/frequency encoding or embeddings for high-cardinality categories (e.g., zip codes) where one-hot would explode dimensionality.

Q29. What is reinforcement learning? Can you give an example of where it could be used?

A paradigm where an agent learns a policy by taking actions in an environment and receiving rewards or penalties, optimizing for cumulative long-term reward rather than a single labeled example. Examples: game-playing AI (AlphaGo), robotics control, and RLHF for aligning LLM outputs to human preferences.

11. Applied Statistics

Q1. Can you explain the Central Limit Theorem and its importance in statistics?

The CLT states that the distribution of sample means approaches a normal distribution as sample size grows, regardless of the underlying population's distribution (given finite variance). It's important because it justifies using normal-distribution-based methods (confidence intervals, hypothesis tests) even when the raw data isn't normally distributed.

Q2. What is the difference between Type I and Type II errors in hypothesis testing?

A Type I error is a false positive — rejecting a true null hypothesis (e.g., concluding an effect exists when it doesn't). A Type II error is a false negative — failing to reject a false null hypothesis (missing a real effect). Lowering one typically raises the other for a fixed sample size.

Q3. Can you explain what is meant by p-value?

The probability of observing data at least as extreme as what you saw, assuming the null hypothesis is true. A small p-value (below your significance threshold, e.g. 0.05) suggests the observed effect is unlikely to be due to chance alone — it is not the probability the null hypothesis is true.

Q4. How do you assess the normality of a dataset?

Visually with a histogram or Q-Q plot (points should fall on a straight line for normal data), and formally with tests like Shapiro-Wilk or Kolmogorov-Smirnov — though with large samples, even trivial deviations can make these tests reject normality, so visual inspection matters too.

Q5. Can you describe the difference between correlation and causation?

Correlation means two variables move together statistically; causation means one variable's change directly produces the change in the other. Correlation can arise from a genuine causal link, a shared confounding cause, reverse causation, or pure coincidence — establishing causation typically requires controlled experiments or careful causal-inference methods.

Q6. How do you handle missing or corrupted data in a dataset? (Statistics)

See the Machine Learning section — same principles apply: diagnose the missingness mechanism, then drop, impute, or flag as appropriate, being careful that the chosen method doesn't bias downstream statistical estimates.

Q7. What are the assumptions required for linear regression? What if some of these assumptions are violated?

Linearity (relationship between X and Y is linear), independence of errors, homoscedasticity (constant error variance), normality of residuals, and no severe multicollinearity. If violated: transform variables or use a different model for non-linearity, use robust/clustered standard errors for heteroscedasticity or non-independence, and use regularization or drop/combine features for multicollinearity.

Q8. Explain the concept of power in statistical tests.

Statistical power is the probability of correctly rejecting a false null hypothesis (i.e., detecting a real effect when one exists) — equivalently, 1 minus the Type II error rate. Higher power requires larger sample sizes, larger true effect sizes, or less noisy measurements.

Q9. How would you explain to a non-technical team member what a confidence interval is?

It's a range of plausible values for the true metric, built from your sample, such that if you repeated the experiment many times, about 95% (for a 95% CI) of such intervals would contain the true value — it communicates the *uncertainty* around your estimate, not just a single number.

Q10. What is multiple regression and when do we use it?

Regression with more than one predictor variable, used when an outcome plausibly depends on several factors simultaneously (e.g., house price depending on size, location, and age) — it lets you estimate each predictor's effect while holding the others constant.

Q11. Can you describe the difference between cross-sectional and longitudinal data?

Cross-sectional data captures many subjects at a single point in time (a snapshot). Longitudinal data tracks the same subjects repeatedly over time, allowing you to observe change and trends within individuals, not just differences across a population at one moment.

Q12. What is the role of data cleaning in data analysis?

It ensures the data is accurate, consistent, and usable before analysis — handling missing values, duplicates, inconsistent formatting, and outliers — because even the best model or analysis produces unreliable conclusions on dirty input data ('garbage in, garbage out').

Q13. Can you explain the difference between ANOVA and t-test?

A t-test compares means between exactly two groups. ANOVA (Analysis of Variance) compares means across three or more groups simultaneously while controlling the overall Type I error rate — running multiple pairwise t-tests instead would inflate the false-positive rate.

Q14. How do you interpret the coefficients of a logistic regression model?

Each coefficient represents the change in the log-odds of the positive class per unit increase in that predictor, holding others constant. Exponentiating a coefficient gives an odds ratio — e.g., $\exp(\beta)=1.5$ means a one-unit increase in that feature multiplies the odds of the positive outcome by 1.5.

Q15. What is multicollinearity and how do you detect and deal with it?

When predictor variables are highly correlated with each other, making individual coefficient estimates unstable and hard to interpret. Detect via correlation matrices or Variance Inflation Factor (VIF); address by removing/combining correlated features, or using regularization (Ridge) that handles it more gracefully.

Q16. Can you describe the difference between a parametric and a nonparametric test?

Parametric tests (t-test, ANOVA) assume the data follows a specific distribution (usually normal) and estimate its parameters. Nonparametric tests (Mann-Whitney U, Kruskal-Wallis) make fewer distributional assumptions, trading some statistical power for robustness when normality doesn't hold.

Q17. What are some of the methods you would use to detect outliers?

Visual methods (box plots, scatter plots), statistical rules (z-score beyond ~3, IQR method — points beyond 1.5x IQR from Q1/Q3), and model-based methods (isolation forest, DBSCAN clustering) for multivariate outliers.

Q18. What is survival analysis and when can it be useful?

A set of methods (Kaplan-Meier estimator, Cox proportional hazards model) for analyzing the time until an event occurs, explicitly handling 'censored' data where the event hasn't happened yet for some subjects by the end of observation. Useful for churn prediction, equipment failure time, and medical time-to-event studies.

Q19. Can you explain what is meant by the term 'residual' in regression analysis?

The residual is the difference between the observed value and the value predicted by the model for that data point. Analyzing residuals (patterns, non-random structure, heteroscedasticity) is how you check whether your regression assumptions actually hold.

Q20. Describe a situation where you used statistical analysis to make a decision or solve a problem. (behavioral)

Structure this with a specific example: the business question, the data/test you ran, the statistical method and why it fit, the result, and the concrete decision or action that followed from it.

12. A/B Testing & Experimentation

Q1. How would you design an experiment to test a new feature?

Define a single primary success metric tied to the business goal, form a clear hypothesis, randomly assign users to control/treatment to avoid confounds, calculate the required sample size upfront for adequate power, run for a full business cycle to average out day-of-week effects, and pre-register the analysis plan to avoid p-hacking.

Q2. Can you explain the concept of A/B testing and how it can be useful for product development?

A/B testing randomly splits users between a control (current) and treatment (new) experience and compares outcomes, isolating the causal effect of the change from confounding factors. It lets product teams validate that a change actually improves the target metric before a full rollout, reducing the risk of shipping something that looks good in theory but hurts in practice.

Q3. Describe a product that you think is particularly well-designed. What makes it effective? (behavioral)

Pick a product you know well and tie your reasoning to concrete design principles — clarity of the core workflow, minimal friction to the 'aha moment', consistent feedback to the user, and how it removes unnecessary choices rather than just listing features you like.

Q4. How would you measure the success of a new feature launch?

Tie measurement to the feature's intended goal — adoption/usage rate, the specific metric it was meant to move (retention, conversion, engagement), and guardrail metrics to ensure it didn't hurt something else (latency, churn, support tickets) — evaluated via a proper A/B test where possible.

Q5. How would you determine the optimal sample size for an A/B test?

Use a power analysis: specify the minimum detectable effect size you care about, your desired significance level (commonly 0.05) and power (commonly 0.8), and the baseline metric's variance — these feed a standard sample-size formula (or calculator) to get the required n per group.

Q6. What factors would you consider before rolling out a new feature?

Statistical significance and practical significance of the effect, guardrail metrics (no regressions elsewhere), engineering/operational readiness (rollback plan, monitoring), segment-level effects (does it help everyone or hurt a subgroup), and business/strategic alignment beyond just the raw metric win.

Q7. How would you interpret the results of an A/B test that showed no significant difference between the control and treatment groups?

First check whether the test was adequately powered to detect a meaningful effect — a null result from an underpowered test is inconclusive, not evidence of no effect. If well-powered, consider that the change may genuinely have no effect, may have offsetting effects across segments, or the chosen metric may not capture the actual impact.

Q8. Can you explain the concept of statistical power in the context of A/B testing?

The probability the test detects a real effect of a given size if one truly exists. Low power means you might miss real improvements (false negatives); it's driven up by larger sample size, larger true effect size, and

lower measurement variance.

Q9. Can you describe a time when you used data to make a decision about a product? (behavioral)

Give a concrete example: the ambiguous decision, the data/experiment you used to resolve it, and the outcome — emphasizing how the data changed (or confirmed) the decision versus intuition alone.

Q10. How would you approach balancing the need for innovation with the potential risk to user experience? (behavioral)

Ship experimental changes to a small controlled slice of traffic first, define clear guardrail metrics that trigger an automatic rollback, and separate 'exploration' surfaces (opt-in beta) from core flows that need stability — so innovation doesn't come at the cost of the core experience.

Q11. What is a p-value? How would you explain it to a non-technical stakeholder? (duplicate)

See the Statistics section above — practically: 'if there were truly no real difference, a result this extreme would only happen this often by chance.'

Q12. Describe a situation where A/B testing would not be appropriate.

When sample size/traffic is too small to reach significance in a reasonable time, when the change affects the whole system non-severably (e.g., a backend migration with no meaningful control group), when network/social effects mean treatment 'leaks' into control (e.g., social features), or when ethical/safety concerns make withholding a feature from a control group inappropriate.

Q13. What metrics would you look at to understand user engagement for a mobile app?

DAU/MAU (and their ratio, 'stickiness'), session length and frequency, retention curves (D1/D7/D30), feature adoption rate, and funnel completion rates for key flows — chosen based on what 'engaged' actually means for that specific app's value proposition.

Q14. What are some pitfalls or common mistakes in A/B testing?

Peeking at results early and stopping as soon as significance appears (inflates false positives), not correcting for multiple comparisons when testing many metrics, novelty effects skewing short-term results, sample ratio mismatch (unequal group sizes indicating a bug), and network interference between control and treatment users.

Q15. How would you prioritize features for a new product?

Frameworks like RICE (Reach, Impact, Confidence, Effort) or a value-vs-effort matrix, informed by user research/data on pain points, alignment with strategic goals, and technical dependencies — prioritizing high-impact, low-effort items first while sequencing dependencies correctly.

Q16. How do you deal with seasonality in A/B testing?

Run the test across a full cycle (e.g., a full week or more, covering weekday/weekend patterns) rather than a short window, and if seasonal effects are strong, compare treatment-vs-control effect size rather than raw absolute metrics, since seasonality affects both groups roughly equally.

Q17. Describe how you would validate the results of an A/B test.

Check for sample ratio mismatch (did randomization actually produce equal-sized groups), confirm the effect is both statistically and practically significant, examine segment-level consistency (not driven by one outlier segment), and where feasible, replicate on a fresh traffic slice before fully committing.

Q18. How would you design a recommendation system for a new product?

Start with simple, robust baselines (popularity-based, content-based similarity) to handle cold-start, add collaborative filtering as interaction data accumulates, evaluate offline (precision@k, NDCG) then online via A/B test on business metrics (CTR, conversion, retention), and continuously retrain as user behavior shifts.

Q19. How would you use data to improve an existing product's user experience?

Analyze funnels to find drop-off points, segment behavior to see where friction concentrates, form specific hypotheses about the cause, test fixes via A/B experiments rather than shipping blind, and monitor guardrail metrics post-launch to confirm the fix didn't create new friction elsewhere.

Q20. What could be some potential issues with running multiple A/B tests at the same time?

Interaction effects between overlapping experiments can confound results, increased chance of false positives across many simultaneous tests (multiple comparisons problem), and infrastructure complexity in correctly attributing metric movements to the right experiment — mitigated with orthogonal experiment design/traffic allocation and careful statistical correction.

13. SQL & Databases

Q1. How can you combine data from multiple tables in SQL? What is the difference between SQL and NoSQL databases?

Combine tables with JOINs (INNER, LEFT, RIGHT, FULL) on a shared key. SQL databases are relational, with fixed schemas and strong ACID guarantees, best for structured data with complex relationships. NoSQL databases (document, key-value, column, graph) offer flexible schemas and horizontal scalability, trading some consistency guarantees for performance/flexibility on unstructured or high-volume data.

Q2. What is a relational database? What about a non-relational database?

A relational database organizes data into tables with predefined schemas and relationships enforced via keys, queried with SQL (e.g., Postgres, MySQL). A non-relational (NoSQL) database stores data more flexibly — as documents, key-value pairs, wide columns, or graphs — often prioritizing scalability and schema flexibility over strict relational consistency (e.g., MongoDB, DynamoDB, Neo4j).

Q3. Can you explain the concept of database normalization?

Organizing tables to reduce data redundancy and avoid update/insert/delete anomalies, by progressively splitting data into related tables (1NF, 2NF, 3NF, ...) based on functional dependencies — e.g., storing a customer's address once in a customers table rather than repeating it on every order row.

Q4. What is a primary key in SQL? How does it differ from a foreign key?

A primary key uniquely identifies each row in its own table (not null, unique). A foreign key is a column in one table that references a primary key in another table, enforcing referential integrity between the two.

Q5. What is a transaction in SQL? Can you describe a scenario where you would need to use transactions?

A transaction is a group of operations executed as a single atomic unit — either all succeed or all roll back. Example: transferring money between two accounts requires debiting one and crediting the other atomically, so a failure midway never leaves money 'lost' or duplicated.

Q6. Can you describe how indexing works in SQL? Why is it important for database performance?

An index is an auxiliary data structure (commonly a B-tree) that lets the database locate rows matching a condition without scanning the entire table, much like a book's index avoids reading every page. It dramatically speeds up reads on large tables at the cost of extra storage and slightly slower writes (since indexes must be updated too).

Q7. What are some strategies you would use to optimize a slow-performing SQL query?

Check the query execution plan (EXPLAIN) for full table scans, add indexes on filtered/joined columns, avoid SELECT *, rewrite correlated subqueries as joins where possible, ensure statistics are up to date, and consider denormalization or caching for read-heavy hot paths.

Q8. Can you explain what ACID properties are in the context of databases?

Atomicity (a transaction fully completes or fully rolls back), Consistency (a transaction moves the database from one valid state to another, respecting constraints), Isolation (concurrent transactions don't interfere with each other's intermediate state), and Durability (once committed, data survives crashes) — the

guarantees that make relational transactions reliable.

Q9. How can denormalization improve the performance of a SQL database?

By intentionally duplicating data or pre-joining tables to avoid expensive joins at read time, trading some redundancy and write complexity for much faster reads — common in read-heavy analytics/reporting systems.

Q10. Can you explain the role of a database management system (DBMS)?

Software that manages storage, retrieval, concurrency control, security, and integrity of data, providing a structured interface (SQL or other query languages) so applications don't need to manage raw file storage or low-level data access themselves.

Q11. What are some common causes of database performance issues and how would you address them?

Missing/poor indexes, unoptimized queries (full scans, N+1 query patterns), lock contention from long-running transactions, insufficient hardware resources, and outdated statistics — addressed via indexing, query rewriting, connection pooling, caching, and scaling (vertical or via replicas/sharding).

Q12. Can you explain the concept of sharding in databases? What problems does it solve?

Splitting a large dataset horizontally across multiple database servers (shards), each holding a subset of rows (e.g., by user ID range). It solves the problem of a single machine's storage/throughput limits, enabling horizontal scalability at the cost of added complexity for cross-shard queries and joins.

Q13. What is an ORM (Object-Relational Mapping)? What are the pros and cons of using an ORM?

A library that maps database tables to application objects/classes, letting developers query/manipulate data using native code instead of raw SQL. Pros: faster development, less boilerplate, some DB-portability. Cons: can generate inefficient queries, abstracts away performance-critical details, and can be harder to debug than hand-written SQL for complex queries.

Q14. Can you explain the CAP theorem in the context of databases?

In a distributed system, you can only guarantee two of three properties at once during a network partition: Consistency (every read gets the latest write), Availability (every request gets a response), and Partition tolerance (the system keeps working despite network splits). Since partitions are unavoidable in practice, real distributed databases choose to favor consistency (CP) or availability (AP) when a partition occurs.

Q15. How does data replication work in SQL databases? What are its benefits?

A primary database's changes are copied to one or more replica servers (synchronously or asynchronously). Benefits: read scalability (route reads to replicas), high availability/failover (promote a replica if the primary fails), and geographic distribution for lower read latency.

Q16. How would you handle concurrency issues in a SQL database?

Use appropriate transaction isolation levels for the workload, apply row-level locking or optimistic concurrency control (version/timestamp checks) to avoid lost updates, and keep transactions short to minimize contention.

Q17. Can you explain the concept of a database lock?

A mechanism that restricts concurrent access to a piece of data (row, table, or page) to prevent conflicting simultaneous modifications, ensuring transaction isolation — but overly broad or long-held locks can cause contention or deadlocks.

Q18. What is a database view and what are its uses?

A virtual table defined by a stored query, presenting data from one or more underlying tables without duplicating it. Used to simplify complex queries, enforce access control (expose only certain columns/rows), and provide a stable interface even if underlying tables change.

Q19. How do you use the LIKE operator in SQL? What is the difference between INNER JOIN and OUTER JOIN in SQL?

LIKE matches string patterns using wildcards (% for any sequence, _ for a single character), e.g. WHERE name LIKE 'A%'. INNER JOIN returns only rows with matches in both tables; OUTER JOIN (LEFT/RIGHT/FULL) also returns unmatched rows from one or both tables, filling in NULLs for the missing side.

Q20. What is the difference between a left join, a right join, and an inner join?

INNER JOIN returns only rows matching in both tables. LEFT JOIN returns all rows from the left table plus matches from the right (NULL where no match). RIGHT JOIN is the mirror image, returning all rows from the right table plus matches from the left.

14. Coding Questions (Python / Pandas)

Q1. Write a Python function to find the factorial of a number.

```
def factorial(n):
if n < 0:
raise ValueError('n must be non-negative')
result = 1
for i in range(2, n + 1):
result *= i
return result
```

Q2. How would you write a function to check if a string is a palindrome in Python?

```
def is_palindrome(s):
cleaned = ''.join(ch.lower() for ch in s if ch.isalnum())
return cleaned == cleaned[::-1]
```

Q3. How can you handle missing values in a dataset using Pandas?

df.isna().sum() # count missing per column
df.dropna() # drop rows with any missing value
df.fillna(df.mean(numeric_only=True)) # impute numeric columns with mean
df['col'].fillna(df['col'].mode()[0]) # impute categorical with mode

Q4. Given an array of integers, find the two numbers that sum up to a specific target number.

```
def two_sum(nums, target):
seen = {}
for i, n in enumerate(nums):
complement = target - n
if complement in seen:
return [seen[complement], i]
seen[n] = i
return None # O(n) time using a hash map
```

Q5. Write a Python function that sorts the elements in a list.

```
def my_sort(lst):
return sorted(lst) # Timsort, O(n log n)

# Or implement quicksort manually:
def quicksort(lst):
if len(lst) <= 1:
return lst
pivot = lst[len(lst)//2]
left = [x for x in lst if x < pivot]
mid = [x for x in lst if x == pivot]
right = [x for x in lst if x > pivot]
return quicksort(left) + mid + quicksort(right)
```

Q6. How do you merge two data frames in Python using Pandas?

merged = pd.merge(df1, df2, on='key_column', how='inner') # or 'left'/'right'/'outer'

Q7. Given a list of numbers, write a function to find the mean, median, and mode.

```
import statistics as st

def summarize(nums):
return {
'mean': st.mean(nums),
'median': st.median(nums),
'mode': st.mode(nums),
}
```

Q8. How do you create a binary variable in Python from a numerical variable?

df['is_high'] = (df['value'] > threshold).astype(int)

Q9. Write a Python function to implement a linear regression model.

```
import numpy as np

def fit_linear_regression(X, y):
X_b = np.c_[np.ones((X.shape[0], 1)), X] # add bias/intercept term
# Normal equation: theta = (X^T X)^-1 X^T y
theta = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y
return theta # theta[0] is intercept, rest are coefficients
```

Q10. Write a function to calculate the Euclidean distance between two points.

```
import math

def euclidean_distance(p1, p2):
return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))
```

Q11. How can you transpose a DataFrame in Pandas?

df_transposed = df.T

Q12. How do you calculate percentiles with numpy?

```
import numpy as np

p90 = np.percentile(data, 90)
p25, p50, p75 = np.percentile(data, [25, 50, 75])
```

Q13. Given a string, write a function to check if it's a permutation of a palindrome.

```
from collections import Counter

def can_permute_palindrome(s):
counts = Counter(ch.lower() for ch in s if ch.isalnum())
odd_counts = sum(1 for c in counts.values() if c % 2 != 0)
return odd_counts <= 1 # at most one character can have an odd count
```

Q14. Write a function to implement the K-nearest neighbors (KNN) algorithm.

```
import numpy as np
from collections import Counter

def knn_predict(X_train, y_train, x_query, k=3):
dists = np.linalg.norm(X_train - x_query, axis=1)
nearest_idx = np.argsort(dists)[:k]
nearest_labels = [y_train[i] for i in nearest_idx]
return Counter(nearest_labels).most_common(1)[0][0]
```

Q15. How would you write a function that performs a SQL-like groupby operation on a Pandas DataFrame?

result = df.groupby('category')['value'].agg(['sum', 'mean', 'count'])

Q16. Given a list of numbers, write a function to find the outliers.

```
import numpy as np

def find_outliers_iqr(data):
q1, q3 = np.percentile(data, [25, 75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
return [x for x in data if x < lower or x > upper]
```

Q17. How can you visualize a decision tree in Python?

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)
plot_tree(clf, feature_names=feature_names, filled=True)
plt.show()
```

Q18. Write a Python function to calculate the root mean square error (RMSE) of a machine learning model.

```
import numpy as np

def rmse(y_true, y_pred):
return np.sqrt(np.mean((np.array(y_true) - np.array(y_pred)) ** 2))
```

Q19. Write a Python function that performs a binary search in a sorted array.

```
def binary_search(arr, target):
lo, hi = 0, len(arr) - 1
while lo <= hi:
mid = (lo + hi) // 2
if arr[mid] == target:
return mid
elif arr[mid] < target:
lo = mid + 1
else:
hi = mid - 1
return -1 # O(log n) time
```

Q20. How do you filter rows in a Pandas DataFrame?

filtered = df[(df['age'] > 30) & (df['city'] == 'NY')]
# or: df.query("age > 30 and city == 'NY'")